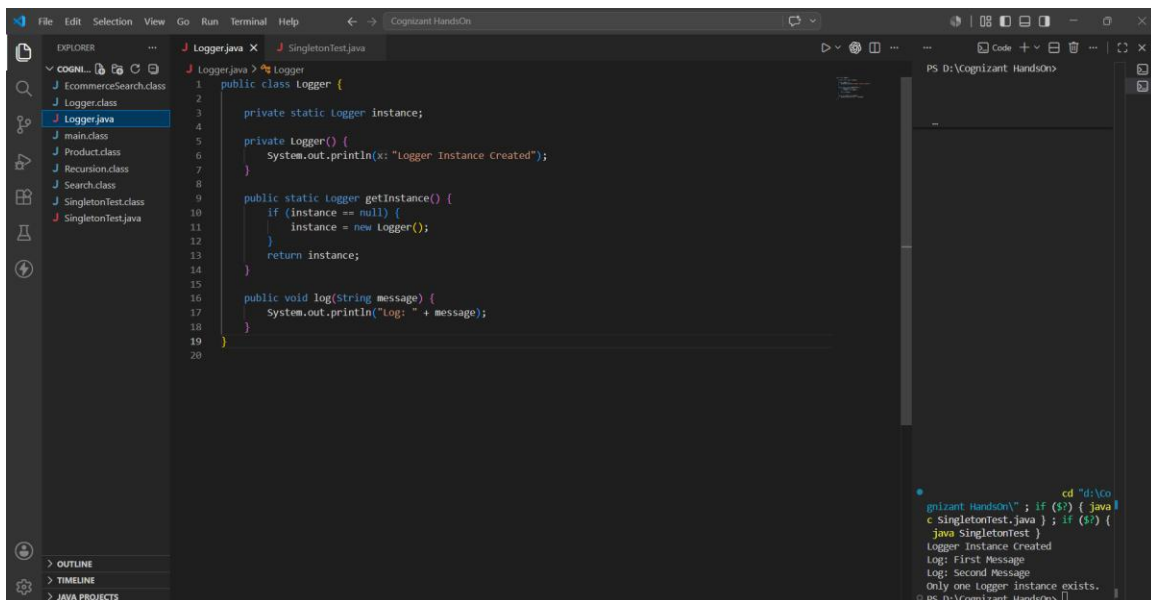


Week 1: Design Pattern and Principles

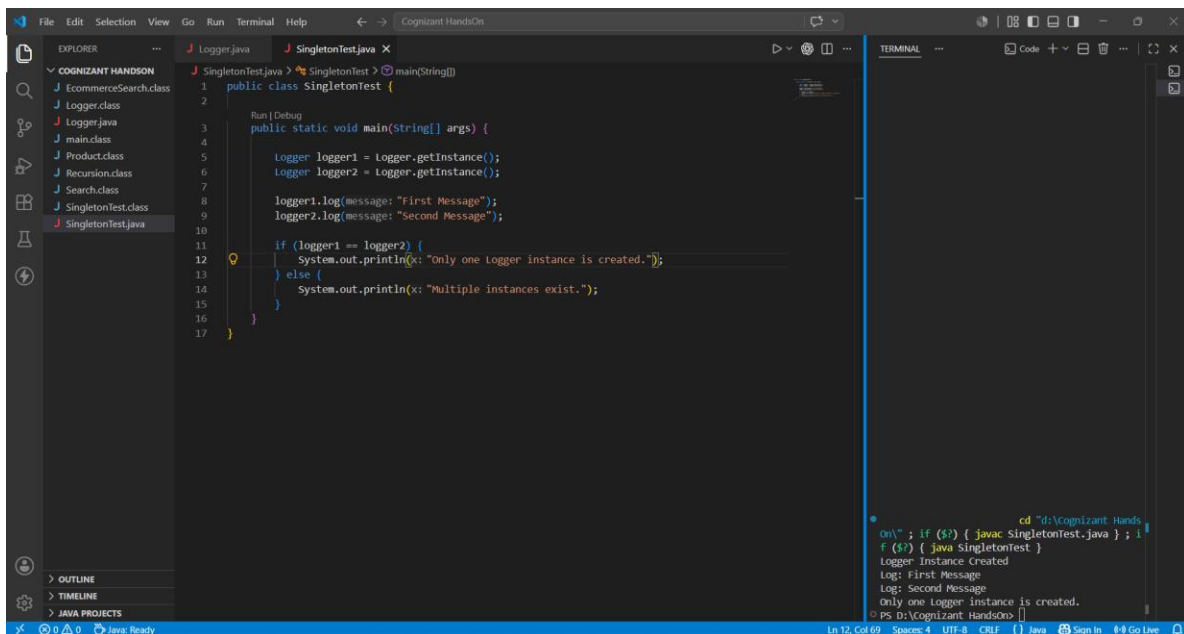
Exercise 1 Solution: Implementing the Singleton Pattern

Logger Class:



```
1 public class Logger {
2
3     private static Logger instance;
4
5     private Logger() {
6         System.out.println("Logger Instance Created");
7     }
8
9     public static Logger getInstance() {
10        if (instance == null) {
11            instance = new Logger();
12        }
13        return instance;
14    }
15
16    public void log(String message) {
17        System.out.println("Log: " + message);
18    }
19 }
20
```

Singleton implementation:



```
1 public class SingletonTest {
2
3     public static void main(String[] args) {
4         Logger logger1 = Logger.getInstance();
5         Logger logger2 = Logger.getInstance();
6
7         logger1.log(message: "First Message");
8         logger2.log(message: "Second Message");
9
10        if (logger1 == logger2) {
11            System.out.println("Only one logger instance is created.");
12        } else {
13            System.out.println("Multiple instances exist.");
14        }
15    }
16 }
17
```

Exercise 2 Solution: Implementing the Factory Method Pattern

Document Interface:

```
interface Document {  
    void open();  
}
```

Word Document:

```
class WordDocument implements Document {  
    public void open() {  
        System.out.println("Opening Word Document");  
    }  
}
```

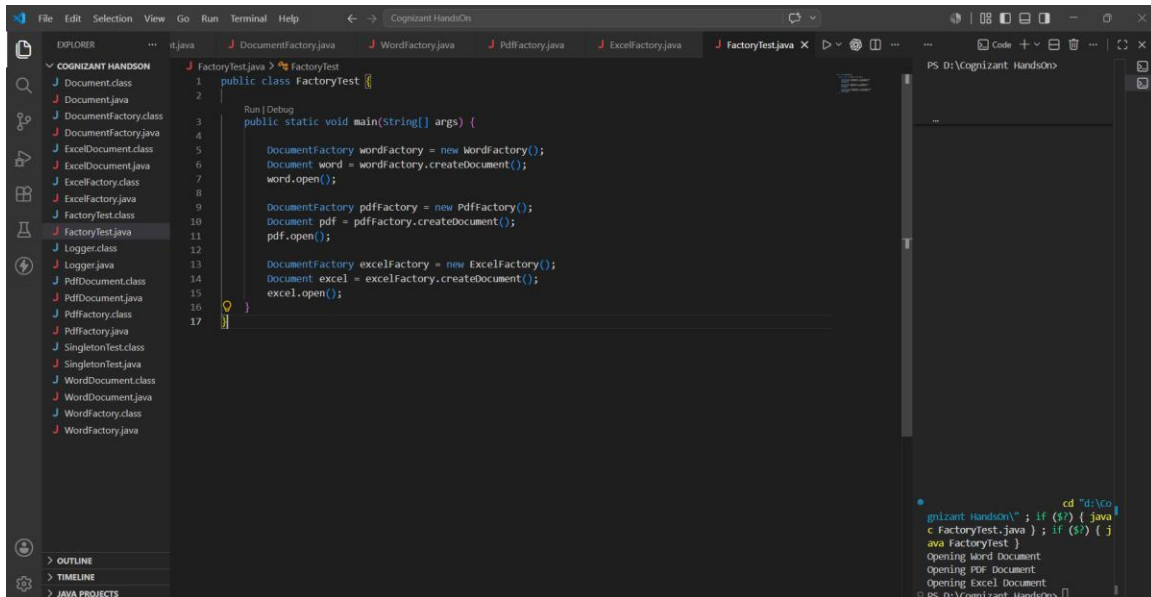
PDF Document:

```
class PdfDocument implements Document {  
    public void open() {  
        System.out.println("Opening PDF Document");  
    }  
}
```

Excel Document:

```
class ExcelDocument implements Document {  
    public void open() {  
        System.out.println("Opening Excel Document");  
    }  
}
```

Factory Classes:



Test Class:

